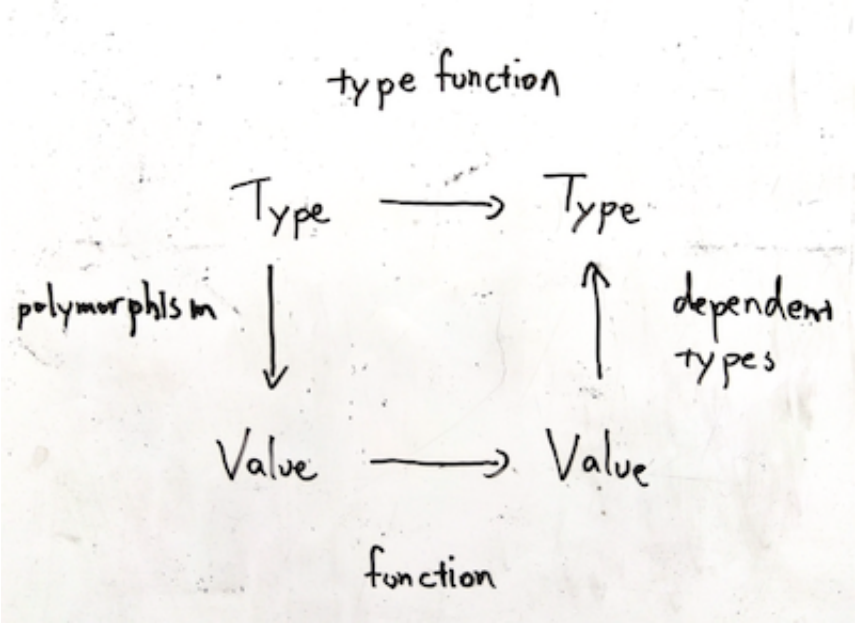
Overview

Lambda cube




```
-- function: Value -> Value
not :: Bool -> Bool
not True = False
not False = True


-- type function: Type -> Type
type Not :: Bool -> Bool
type family Not a where
  Not 'True = 'False
  Not 'False = 'True


-- polymorphism: Type -> Value
show :: Show a => a -> String                    -- polymorphism with typeclass (open function)
show @Int
show @Bool

fromType :: foreach (a :: Bool) -> Bool          -- polymorphism with foreach (closed function)
fromType (type True) = True
fromType (type False) = False


-- dependent type: Value -> Type
toType :: Bool -> Some (foreach (a :: Bool))
toType True = Some (type True)
toType False = Some (type False)
```

Type

Wiki: https://wiki.haskell.org/Algebraic_data_type

Language spec: https://www.haskell.org/onlinereport/haskell2010/haskellch4.html#x10-680004.2

Haskell uses an **algebraic** type system.

i.e. type is a combination of **sum aka compositions** and **catesian product** of other types

```
-- e.g.
data MyType = A                 -- {} i.e. empty set
data MyType a b = A a b         -- product a x b
data MyType a b = A a | B b     -- sum a + b
data MyType a b = A a | B a b   -- a + (a x b)
```

A **type** is defined by a **type constructor** + one or more **data constructors**.
A type constructor consists of a **type constructor name**+ zero or more **type variables**.
A data constructor consists of a **data constructor name** + zero or more **type variables.**
e.g. if
type constructor= MyType a b
type constructor name= MyType with type variables=a b
data constructors= A a | B a b
data constructor name=A with type variables=a
data constructor name=B with type variables=a b

```
data MyType a b = A a | B a b
```

Currying in type constructor
Partially applied type constructors
Like normal functions, type constructors are also **curried**

```
-- e.g. Either is binary
Either :: * -> * -> *
Either :: * -> (* -> *)      -- same i.e. curried
```

Hence can be **partially applied**
i.e. a partially applied type constructor is also a **type themselves.**

```
-- e.g.
Either :: * -> * -> *         -- binary type
Either Int :: * -> *          -- unary type
Either a :: * -> *
Either Int Int :: *           -- nullary type
Either a b :: *
```

Usage examples

```
-- usage: in type signatures
Data.Functor.Compose
 -- type constructor Compose x y z
Data.Functor.Compose $ Right [1] :: Compose (Either a) []
Int       -- x=Either a y=[] z=Int    a=free
Data.Functor.Compose $ Left [1] :: Compose (Either [Int]) a
b       -- x=Either [Int] y=a z=b   y,z=free

-- usage: in class instances
instance Monad ((-> r)) where        -- unary type x ↦ r -> x  i.e.
fixed domain r
```

```
 fmap :: (.)                              -- e.g. Int -> x is a monad
over x
instance Monad (Either a) where          -- unary type x ↦ Either a x
is a monad.
 ...                                       -- e.g. Either Int x is a
monad over x
```

Type constructor != **data constructor**
In OOP, usually type constructor name == data constructor name
OOP types use **type inheritance** so only there is only one data constructor.
i.e. types are **hierarchical**.
Hence no need to use different names.

```
e.g. in C#
class MyType<T> { ... }           type name = MyType<T>
v = new MyType<Int>();           data constructor name = new
MyType<Int>()
```

Haskell types use **type composition** instead of inheritance
i.e. **algebraic sum** of data instead of **hierarchical**.
Hence each type may have more than one data constructor.

```
e.g.
type Bool = True | False
x = True                -- type=Bool, data constructor=True
x = False               -- type=Bool, data constructor=False

type Either a b = Left a | Right b
x = Left 1              -- type=Either Int b, data constructor=Left
x = Right 1             -- type=Either a Int, data
constructor=Right
```

Type declarations

3 ways to declare a type:
1) **algebraic** data type
2) type **synonym**
3) **renamed** type

Declaration 1: Algebraic data type
Spec: https://www.haskell.org/onlinereport/haskell2010/haskellch4.html#x10-680004.2
GHC extensions: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/types.html

Data constructor
A **constructor** of a new data.
n-ary data constructor = object with **n properties.**
3 data constructor formats:
- **prefix constructor**: the most common constructor

```
data MyType a = A a
x = A 1
```

- **infix constructor**: data instantiation via infix
MUST have exactly 2 variables

```
-- e.g. list constructor (:)
x = 1:[2,3]            -- [1,2,3]

data MyType a = a :<> a
x = 1 :<> 1
```

- **labeled fields**: data with **getters** for convenient access

```
data MyType a = A { v1::a, v2::a }
x = A { v1=1, v2='a' }
a = v1 x          -- 1
b = v2 x          -- 'a'
A a b = x         -- same without getters
A a _ = x
A _ b = x
```

Declaration 2: Type synonym
Spec: https://www.haskell.org/onlinereport/haskell2010/haskellch4.html#x10-680004.2

Type synonym
i.e. type **alias**.
Same as **typedef** in C
Alias is exactly the **same** as the original type
i.e. can apply same methods
e.g.

```
type Height = Int
1 :: Int
1 :: Height               -- same
(1::Height) + (1::Int)    -- OK
```

Declaration 3: renamed type
Spec: https://www.haskell.org/onlinereport/haskell2010/haskellch4.html#x10-680004.2

renamed type
Conceptually a **wrapper** of a **single value.**
Hence:
- MUST be exactly **one value** i.e. no algebraic sum
- MUST have a **data constructor**

e.g. renamed types

```
-- Data.Functor.Compose
newtype Compose f g a = Compose { getCompose :: f (g a) }
```

```
  deriving ( Data, Generic, Generic1, Semigroup, Monoid )

-- Control.Monad.State.Lazy
newtype StateT s (m :: Type -> Type) a = StateT (s -> m (a,s))
```

rename type vs data type
Ref: http://wiki.haskell.org/Newtype
Haskell type system uses **wrapper types** to add functionality
e.g. monad transform
Renamed type is an **optimization** for wrapper types by automatically **unwrapping** at compile time.
i.e. the wrapper is the **same** as the wrapped value at runtime.
e.g.

```
newtype MyType a = A a
A 1                    -- same as 1 at runtime
```

Implications
- **performance**: no wrapper overhead
- value **coercion**
e.g. can automatically **unwrap** using **coercion**.

```
{-# LANGUAGE TypeApplications #-}
import Data.Coercion

newtype A a = A a
newtype B b = B b
x = A 1
y = B x

-- e.g. unwrap with coerce
coerce @(A Int) @Int x        -- 1
coerce @(B (A Int)) @Int y    -- 1
```

- inherit **classes** (GHC extension): new wrapper can inherit class from the wrapped inner value.
GHC: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/newtype_deriving.html#extension-GeneralisedNewtypeDeriving

Simplified subsumption
Simplified subsumption
Was simplified subsumption worth it: https://www.reddit.com/r/haskell/comments/ujpzx3/was_simplified_subsumption_worth_it_for_industry/
Was simplified subsumption worth it: https://discourse.haskell.org/t/r-haskell-was-simplified-subsumption-worth-it-for-industry-haskell/4486
[GHC] Simplify Subsumption: https://github.com/ghc-proposals/ghc-proposals/blob/master/proposals/0287-simplify-subsumption.rst
Stricter rules for **forall**.

a) no **deep skolemization**

s :: ((forall a. a -> a) -> Bool) -> String
t :: (Int -> Int) -> Bool

s t :: String — ERROR with simplified subsumption
s (\h -> t h) :: String — OK with expansion
  forall a. h :: a -> a

Error

```
• Couldn't match type: Int -> Int
                  with: forall a. a -> a
  Expected: (forall a. a -> a) -> Bool
    Actual: (Int -> Int) -> Bool
• In the first argument of ‘s’, namely ‘t’
  In the expression: s t
  In an equation for ‘x’: x = s t
```

b) No **deep instantiation**

TODO

c) No **contravariance**

u :: (forall a b. a -> b -> b) -> c
v :: forall a. a -> (forall b. b -> b)

u v :: c — ERROR with simpilfied subsumption
u (\i-> v i) :: c — OK with expansion
  i :: a

Error

```
• Couldn't match type: forall b1. b1 -> b1
                  with: b -> b
  Expected: a -> b -> b
    Actual: a -> forall b. b -> b
• In the first argument of ‘u’, namely ‘v’
  In the expression: u v
  In an equation for ‘y’: y = u v
```

d) No covariance

TODO

==========

Kind

Kind
Language spec: https://www.haskell.org/onlinereport/haskell2010/

haskellch4.html#x10-630004.1
Haskell wiki: https://wiki.haskell.org/Kind
Type of type
By default there is only one **base kind Type (*)** and every type has this kind.
There are 3 types of kinds:
a) kind of **monomorphic type**: Type (*)
e.g. type Int :: Type

b) kind of **polymorphic type**:
e.g. type Maybe :: Type -> Type

c) **Polymorphic kind**: i.e. a **variable**
e.g. type Proxy :: forall k. k -> Type

```
-- Base kind
--
type Int :: Type
type Int :: *                                   Haskell98 notation

type () :: Type
type Bool :: Type
type Char :: Type
type (Maybe Int) :: Type

-- user defined
type MyType :: Type
data MyType = MyType


-- Kind of polymorphic type
--
type Maybe :: Type -> Type                      Unary
type Either :: Type -> Type -> Type             Binary
-- operators
type (+) :: Type -> Type -> Type

-- special
type (,) :: Type -> Type -> Type
type [] :: Type -> Type
type (->) :: Type -> Type

-- user defined
type MyType :: Type -> Type -> Type
data MyType a b = MyType a b
type (<>) :: Type -> Type -> Type
data a <> b = a :<> b


-- Polymorphic kind
--
type Proxy :: k -> Type
type Proxy :: forall k. k -> Type
Proxy (Int :: Type)
Proxy (Maybe :: Type -> Type)

-- Data.Functor
type Sum :: forall k. (k -> Type) -> (k -> Type) -> k -> Type
data Sum = InL (f a) | InR (g a)

type Compose :: forall a b.
  (b -> Type) -> (a -> b) -> a -> Type
data Compose = Compose (f (g a))
```

e.g. check the kind in the REPL
With StarIsType

```
ghci> :k Int
Int :: *
ghci> :k Maybe
Maybe :: * -> *
ghci> import Data.Proxy
ghci> :k Proxy
Proxy :: k -> *
```

Without StarIsType + explicit forall

```
ghci> :set -XNoStarIsType -fprint-explicit-foralls
ghci> :k Int
Int :: Type
ghci> :k Maybe
Maybe :: Type -> Type
ghci> import Data.Proxy
ghci> :k Proxy
Proxy :: forall {k}. k -> Type
```

DataKind
Allows defining a **custom kind** other than the base **Type (*)**

type 'True :: Bool	'True is of **kind** Bool
type 'False :: Bool	
type 'True :: Type	NO. Custom kind isn't a Type

e.g.

```
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE DataKinds #-}

import Data.Kind

-- Alpha type
type Alpha :: Type
data Alpha = A | B

-- Alpha kind
type V1 :: Alpha              -- type 'A :: Alpha
type V1 = 'A

type V2 :: Alpha              -- type 'B :: Alpha
type V2 = 'B


-- e.g. usage
```

```
type AlphaProxy :: Alpha -> Type       -- Alpha kind
data AlphaProxy a = MkAlpha

type NormalProxy :: Type -> Type
data NormalProxy  a = MkNormal

MkAlpha @'A :: AlphaProxy 'A           -- OK
MkAlpha @'B :: AlphaProxy  'B          -- OK
MkAlpha @Int                           -- ERROR

MkNormal @Int :: NormalProxy Int       -- OK
mkNormal @'A                           -- ERROR
```

==========

OLD

Type system of type system i.e. **signature** of **type constructors**
e.g.

Int	kind Type
Char	kind Type
Maybe	kind Type -> Type
Either	kind Type -> Type
Proxy	kind forall a. a-> Type i.e. **polymorphic kind**

e.g. check the kind in the REPL
With StarIsType

```
ghci> :k Int
Int :: *
ghci> :k Maybe
Maybe :: * -> *
ghci> import Data.Proxy
ghci> :k Proxy
Proxy :: k -> *
```

Without StarIsType + explicit forall

```
ghci> :set -XNoStarIsType -fprint-explicit-foralls
ghci> :k Int
Int :: Type
ghci> :k Maybe
Maybe :: Type -> Type
ghci> import Data.Proxy
ghci> :k Proxy
Proxy :: forall {k}. k -> Type
```

Kinds have 3 possible values:
1) **Type (*)** : a nullary type

2) **unspecified variable a**: a **polymorphic kind** type i.e. a param which can be any of the other kinds
3) **a -> b** where a,b are any of the other kinds: a **n-ary type constructor**
e.g.

```
- 1) nullary
Int :: *
Int :: Type                          -- with NoStarType
[Int] :: Type
Maybe Int :: Type

-- 2) polykind
-- e.g. in Data.Proxy
Proxy :: k -> Type
Proxy :: forall k. k -> Type         -- with ExplicitForall

- 3) n-ary type constructor
(,,) :: Type -> Type -> Type                                        --
 binary
(Int,) :: Type -> Type                                              --
 unary
-- e.g. in Data.Functor.Sum
Sum :: forall k. (k -> Type) -> (k -> Type) -> k -> Type
-- ternary with a polykind param k
-- e.g. in Data.Functor.Compose
Compose :: forall k k1. (k1 -> Type) -> (k -> k1) -> k -> Type
 -- ternary with polykind params k1,k2
```